

HashSet - Complete Notes

Simple revision notes written in the way we discussed in chat.

This chapter explains HashSet from zero to interview level with examples, diagrams, internal working, and common mistakes.

1. Why was HashSet created?

HashSet was created to store only unique elements. If you keep checking duplicates in a normal list, the search can become slow because you may need to scan one by one. HashSet solves this by using hashing and fast lookup.

Example: if you are storing usernames, you do not want the same username twice. HashSet is a very good fit for that kind of job.

Main idea
List can allow duplicates
HashSet keeps only unique elements
Fast duplicate checking using hashing

2. What is HashSet?

HashSet is a Set implementation in Java. A Set does not allow duplicates. HashSet does not use index-based access like ArrayList. You add an element, check if it exists, or remove it based on the element itself.

Example:

```
Set<String> set = new HashSet<>();
set.add("Rahul");
set.add("Amit");
set.add("Rahul"); // duplicate, will not be added
```

Result:

[Rahul, Amit]

A very important note: HashSet does not guarantee insertion order. Do not expect the printed order to stay the same as the add order.

3. Real-life analogy

Imagine a classroom where each student name should appear only once on the attendance sheet. If a name is already written, you do not write it again. That is exactly the idea of HashSet.

One-line memory
HashSet = unique items only
No duplicates
No index access
No guaranteed order

4. Internal working

Internally HashSet uses HashMap. This is one of the most important things to remember. The value in HashSet is not really important. The actual work happens through HashMap.

Conceptually:

```
HashSet  
-> HashMap
```

```
Your element becomes the key  
A dummy object called PRESENT becomes the value
```

So when you add an element to HashSet, Java stores it as a key in HashMap and uses a dummy value. Since HashMap does not allow duplicate keys, HashSet automatically gets duplicate prevention.

5. How add() works

Flow:

```
set.add("Rahul")  
->  
map.put("Rahul", PRESENT)  
->  
hashCode()  
->  
bucket  
->  
equals()  
->  
already present?  
->  
yes: ignore  
->  
no: add node
```

First Java uses hashCode() to find the bucket. Then it uses equals() to check whether the exact same element is already there. If yes, it does nothing. If no, it adds the element.

Simple add() story
Find the bucket first
Check if the element already exists there
Ignore duplicate
Add only if it is new

6. How contains() works

contains() is also based on hashing. Java does not scan the whole set blindly. It calculates hashCode(), goes to the bucket, and then uses equals() to confirm the element.

Example:

```
set.contains("Rahul")  
->  
hashCode()  
->  
bucket  
->  
equals()  
->  
true or false
```

That is why HashSet contains() is fast on average.

7. Why duplicates are not allowed

A Set by definition stores only unique items. HashSet follows that rule. If you try to add the same element again, Java sees that it already exists and does not create another copy.

Example:

```
set.add("A")
set.add("B")
set.add("A")
```

Final set:
[A, B]

This is not because HashSet does something magical separately. It happens because the underlying HashMap does not allow duplicate keys.

8. hashCode() and equals() in HashSet

HashSet also depends on hashCode() and equals(). hashCode() helps choose the bucket. equals() confirms whether the element is already present.

Short meaning:

```
hashCode() -> bucket
equals()    -> exact match
```

If you use custom objects in HashSet, you usually need to override both methods. Otherwise two objects that look equal by business logic may still be treated as different.

Golden rule for custom objects
Override equals()
Override hashCode()
Keep both consistent

9. Custom object example

```
class Student {
    int id;
    String name;

    @Override
    public boolean equals(Object o) { ... }

    @Override
    public int hashCode() { ... }
}
```

If two Student objects have the same id and name and should be treated as the same student, then HashSet should not allow both. For that to work properly, equals() and hashCode() must be consistent.

10. HashSet vs ArrayList

HashSet	ArrayList
No duplicates	Duplicates allowed
No index access	Index access available
No guaranteed order	Maintains insertion order
contains() average $O(1)$	contains() $O(n)$

Use ArrayList when order and index matter. Use HashSet when uniqueness matters.

11. HashSet vs TreeSet

HashSet	TreeSet
Unique elements	Unique + sorted elements
No order guarantee	Sorted order
Average $O(1)$	$O(\log n)$
Uses HashMap	Uses TreeMap/Red-Black Tree idea

If you want speed and uniqueness, use HashSet. If you want sorted unique values, use TreeSet.

12. Time complexity

Operation	Average	Reason
add()	$O(1)$	Hashing
contains()	$O(1)$	Direct bucket lookup
remove()	$O(1)$	Same hashing mechanism
Worst case	$O(n)$	Many collisions in one bucket

13. Common mistakes

A common mistake is expecting HashSet to preserve insertion order. It does not guarantee that. Another mistake is thinking HashSet has its own storage engine. It actually uses HashMap internally. People also forget to override equals() and hashCode() for custom objects.

Also remember: HashSet does not use index-based access. You cannot say `set.get(0)` the way you do with a list.

14. Useful methods

Common HashSet methods:

```
add(element)
contains(element)
remove(element)
size()
isEmpty()
clear()
```

These methods are enough for most interview and practical use cases.

15. Interview questions

What is HashSet? Why does it not allow duplicates? How does it work internally? Why does HashSet use HashMap? Why override equals() and hashCode()? What is the difference between HashSet and ArrayList? What is the difference between HashSet and TreeSet?

16. Final revision sheet

Topic	Simple meaning
HashSet	Unique elements only
Internal engine	HashMap
Element storage	Element becomes key
Value storage	Dummy PRESENT object
add()	Adds only if new
contains()	Fast search using hashing
Duplicates	Not allowed
Order	Not guaranteed
Custom objects	Override equals() and hashCode()

One-line memory: HashSet = HashMap wrapper for unique elements. hashCode() finds the bucket, equals() checks the exact element.